

Insertion Sort & Merge Sort

TC1031 · Data Structures and Fundamental Algorithms

Dr Obed Noe Samano Abonce

Last time

Bubble sort and selection sort.

Both $\Theta(n^2)$. But there are differences.

Today: one more $\Theta(n^2)$ sort — and one algorithm that is not $\Theta(n^2)$ at all.

Today

- **Insertion sort.** Still quadratic. Still built by comparing and moving one element at a time.
- **Merge sort.** A different strategy entirely: split the problem before you solve it.

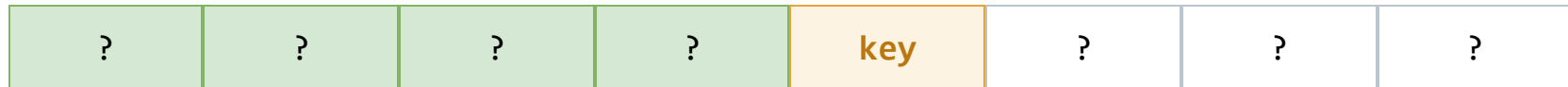
Same working array as last class, so you can compare the counts directly: 42 17 5 30 8 29 3 15

Insertion sort · the idea

- Keep a **sorted zone** at the front. It starts with just one element — a single element is trivially sorted.
- Take the next element — the **key**.
- Slide it left, past everything in the sorted zone that is bigger than it.
- Stop as soon as you meet something smaller, or run out of zone. Drop the key there.

It is how most people sort a hand of playing cards: pick up the next card, slide it into place among the cards you are already holding sorted.

The invariant



Before step i begins, $a[0 \dots i-1]$ is already sorted.

This is *not* the same claim selection sort made. Selection's first k elements were **final** — never touched again. Insertion's sorted zone is sorted *among itself*, but a later key can still slide in front of any of these elements and push them one seat over.

Step by step · $i = 1$

BEFORE · SORTED ZONE = [42]



key = 17 · compare to $a[0] = 42 \rightarrow 42 > 17$, shift it right · **1 comparison, 1 shift**

AFTER



Step by step · $i = 2$

BEFORE · SORTED ZONE = [17, 42]



key = 5 · 42 and 17 are both greater → both shift right · **2 comparisons, 2 shifts**

AFTER



Step by step · $i = 3$

BEFORE · SORTED ZONE = [5, 17, 42]



key = 30 · 42 is greater and shifts right; 17 is not → stop · **2 comparisons, 1 shift**

AFTER



Step by step · i = 4

BEFORE · SORTED ZONE = [5, 17, 30, 42]



key = 8 · 42, 30 and 17 are greater and shift right; 5 is not → stop · **4 comparisons, 3 shifts**

AFTER



Step by step · $i = 5$

BEFORE · SORTED ZONE = [5, 8, 17, 30, 42]



key = 29 · 42 and 30 are greater and shift right; 17 is not → stop · **3 comparisons, 2 shifts**

AFTER



Step by step · $i = 6$, the big one

BEFORE · SORTED ZONE = [5, 8, 17, 29, 30, 42]



key = 3 · 42, 30, 29, 17, 8, 5 are all greater → every one of them shifts right · **6 comparisons, 6 shifts**

AFTER



The key does not stop until it has passed everything bigger than it. This is what it costs when an element starts far from where it belongs.

Step by step · $i = 7$

BEFORE · SORTED ZONE = [3, 5, 8, 17, 29, 30, 42]



key = 15 · 42, 30, 29 and 17 are greater and shift right; 8 is not → stop · **5 comparisons, 4 shifts**

AFTER



All eight elements are now in their final sorted order.

The full run

i	key	comparisons	shifts
1	17	1	1
2	5	2	2
3	30	2	1
4	8	4	3
5	29	3	2
6	3	6	6
7	15	5	4
Total		23	19

19 shifts. The exact same number as bubble sort's swaps, on this same array.

Not a coincidence: both quantities count the same thing — the number of inversions in the array. Bubble fixes one inversion per swap; insertion fixes one per shift.

A fair comparison

A swap touches two array cells. A shift touches one — Sedgewick calls it a “**half exchange**.” To compare algorithms by cost, not just by name, count the same thing for all of them: writes to the array.

	Its own count	Array writes
Bubble sort	19 swaps	38
Selection sort	5 swaps	10
Insertion sort	19 shifts + 7 placements	26

Comparisons decide the complexity class — $\Theta(n^2)$ for all three, no matter how each one moves data. What decides who is actually faster, once two algorithms already share that class, is **the real cost of everything they do** — comparisons and writes together, weighted by how expensive each is for your data. That is why insertion sort is not tied with bubble sort here: counted this way, it lands between selection and bubble — the “19 = 19” on the last slide was a coincidence about this array's inversions, not a statement about cost.

Insertion sort · code

```
insertionSort.cpp

void insertionSort(int a[], int n) {
    for (int i = 1; i < n; i++) {
        int key = a[i];
        int j = i - 1;
        while (j >= 0 && a[j] > key) {
            a[j + 1] = a[j];
            j--;
        }
        a[j + 1] = key;
    }
}
```

There is no swap. The bigger element just slides one seat to the right; the key is written exactly once, when the sliding stops.

Insertion sort • complexity

Pass i compares the key against the i elements already in the sorted zone — at most i comparisons, if it has to travel all the way to the front.



worst case — reverse-sorted input, every key travels all the way to the front

$$1 + 2 + \dots + (n-1) = n(n-1)/2 \rightarrow \Theta(n^2)$$

Best case — already sorted: every key stops after exactly one comparison. $(n-1) \times 1 = n-1 \rightarrow O(n)$

Best case

Same eight values, already sorted. Run insertion sort again.

	Comparisons	Case
This run (above)	23	worst case, $\Theta(n^2)$
Already sorted	7	best case, $O(n)$

Every key looks left, finds something already smaller, and stops immediately — exactly one comparison per key, zero shifts.

Same idea as bubble sort's flag: a good algorithm should notice when the input is already close to done. This is why insertion sort is still used inside real sorting libraries, on small or nearly-sorted stretches of a larger array.

See it live

An interactive walkthrough of insertion sort — one comparison at a time.

Open the walkthrough →

noe-tec.github.io/data-structures/01-fundamentals/pages/sorting-walkthrough.html

- Tabs for **bubble**, **selection** and **insertion** sort.
- Presets: already sorted, reversed, with duplicates, one out of place.
- Step forward and back one comparison at a time — or finish an entire pass at once.

Try the **already sorted** preset on the Insertion tab — it is exactly the best case you just saw.

Where insertion sort runs out of tricks

It is still comparing and shifting **one element at a time** through a zone it built one element at a time.

What if, instead of scanning, we **split** the problem before solving it?

Merge sort · the idea

- **Divide.** Split the array in half.
- **Conquer.** Sort each half — the same way. Recursively.
- **Combine.** Merge the two sorted halves into one sorted array.

The base case: an array of one element is already sorted. There is nothing left to divide.

A strategy that calls itself on a smaller version of the same problem is called **divide and conquer**.

Divide, all the way down

Original array



Split 1



Split 2



Singles — nothing left to split



$8 \rightarrow 4 \rightarrow 2 \rightarrow 1$. Three splits. In general, $\log_2 n$ splits.

The merge mechanism

Two sorted piles. One output.

- Look at the front of each pile.
- Take the **smaller** one. Move it to the output.
- Repeat until one pile is empty.
- Copy whatever is left in the other pile — it is already in order.

That is the entire idea. Merge sort is this operation, applied over and over, on bigger and bigger piles.

Merging the singles

merge [42] + [17]

Compare	Smaller	Output so far
42 vs 17	17	17
left pile remains	42	17, 42

merge [5] + [30]

Compare	Smaller	Output so far
5 vs 30	5	5
right pile remains	30	5, 30

Every merge of two singles takes exactly one comparison.

One level up

merge [17, 42] + [5, 30]

Compare	Smaller	Output so far
17 vs 5	5	5
17 vs 30	17	5, 17
42 vs 30	30	5, 17, 30
left pile remains	42	5, 17, 30, 42

Four elements, three real comparisons — one pile always finishes before the other, so the last step is a free copy.

The right half, the same way

[8] with [29], and [3] with [15], give [8, 29] and [3, 15]. One level up:

merge [8, 29] + [3, 15]

Compare	Smaller	Output so far
8 vs 3	3	3
8 vs 15	8	3, 8
29 vs 15	15	3, 8, 15
left pile remains	29	3, 8, 15, 29

The final merge

merge [5, 17, 30, 42] + [3, 8, 15, 29]

Compare	Smaller	Output so far
5 vs 3	3	3
5 vs 8	5	3, 5
17 vs 8	8	3, 5, 8
17 vs 15	15	3, 5, 8, 15
17 vs 29	17	3, 5, 8, 15, 17
30 vs 29	29	3, 5, 8, 15, 17, 29
left pile remains	30	3, 5, 8, 15, 17, 29, 30
left pile remains	42	3, 5, 8, 15, 17, 29, 30, 42

Sorted.

Merge sort · see it live

Once the code on the next slide makes sense, two interactive tools are waiting for you.

Open the visualizations →

noe-tec.github.io/data-structures

- **Step by step — Merge sort & Quicksort** — divide, merge and partition one step at a time.
 - **Merge sort — recursion tree, step by step** — the call stack as a tree: dive left first, merge back up, one call at a time.
-

Merge sort · code

mergeSort.cpp

```
void mergeSort(int a[], int lo, int hi) {
    if (lo >= hi) return;           // 0 or 1 element: already sorted
    int mid = (lo + hi) / 2;
    mergeSort(a, lo, mid);
    mergeSort(a, mid + 1, hi);
    merge(a, lo, mid, hi);
}
```

merge.cpp

```
void merge(int a[], int lo, int mid, int hi) {
    int temp[hi - lo + 1];
    int i = lo, j = mid + 1, k = 0;
    while (i <= mid && j <= hi)
        temp[k++] = (a[i] <= a[j]) ? a[i++] : a[j++];
    while (i <= mid) temp[k++] = a[i++]; // left pile remains
    while (j <= hi) temp[k++] = a[j++]; // right pile remains
    for (int x = 0; x < hi - lo + 1; x++) a[lo + x] = temp[x];
}
```

mergeSort calls itself on each half, then hands both already-sorted halves to merge. merge builds the output in a temporary array — comparing the two piles, then flushing whichever one is left — and copies it back into a.

lo and **hi** are both valid indices here — hi is the *last* element of the range, not one past it. The first call on our 8-element array is mergeSort(a, 0, 7), covering a[0..7].

Why $O(n \log n)$

Merging level	Merges	Real comparisons
Pairs of singles → pairs	4	4
Pairs → groups of four	2	6
Groups of four → final array	1	6
Total		16

Every level merges the whole array once, at a cost of at most $n-1$ comparisons — here, always well under 7.

There are $\log_2 n$ levels of merging. Each level costs $O(n)$. Total work: $O(n \log n)$ — every time, regardless of how the array started.

Merge sort · the recurrence

Every recursive algorithm's cost can be written as a recurrence: **recursive case** = (# of calls) × **T(reduced size)** + **extra work**.

mergeSort makes **2** calls, each on **$n/2$** elements, plus **$O(n)$** of extra work to merge:

$$T(n) = 2 \cdot T(n/2) + O(n)$$

Several recursive calls — the Master Theorem box: $T(n) = a \cdot T(n/b) + O(n^d)$ — compare a to b^d

Compare	Result	
$a > b^d$	$O(n^{\log_b a})$	
$a = b^d$	$O(n^d \cdot \log n)$	← merge sort
$a < b^d$	$O(n^d)$	

Here: $a = 2$, $b = 2$, $d = 1 \rightarrow b^d = 2^1 = 2 = a$ — the middle case applies.

$$O(n \log n)$$

What that buys you

For an array of one million elements:

Comparisons, roughly	
$\Theta(n^2)$ — bubble, selection, insertion	1,000,000,000,000
$O(n \log n)$ — merge sort	20,000,000

About 50,000× fewer comparisons.

This is what changing the order of growth actually means.

Stability

Two piles, a tie at the front of each.

l e f t : 4^A ... r i g h t : 4^B ...

The rule is **take left when equal**: compare with $\leq$, not $<$.

o u t p u t : 4^A, 4^B, ...

4^A came from the left pile, which held everything that started to the left in the original array. It is placed first — the original order survives.

Merge sort is stable. It never has a reason to let a later equal element pass an earlier one.

Four algorithms, one table

	Worst case	Best case	Stable?
Bubble sort	$\Theta(n^2)$	$O(n)$	yes
Selection sort	$\Theta(n^2)$	$\Theta(n^2)$	no
Insertion sort	$\Theta(n^2)$	$O(n)$	yes
Merge sort	$\Theta(n \log n)$	$\Theta(n \log n)$	yes

Three ways of scanning the array over and over. One way of splitting it. Only the split changes the order of growth.

What to take away

- Insertion sort is still $\Theta(n^2)$ — but its best case is real, and it is why hybrid sorting libraries still use it on small or nearly-sorted stretches.
 - Its shift count equals the number of inversions — the same quantity bubble sort's swaps measured.
 - Divide and conquer does not just shrink a constant. It changes the order of growth itself.
 - Merge sort always costs $O(n \log n)$ — there is no bad input that pushes it to n^2 .
 - That guarantee is not free: merge needs extra space to build its output.
-

Next time

Quicksort. Another divide-and-conquer algorithm.

Same idea — split, recurse, combine — but this time the split is decided by a **value**, not by position.

And its worst case is not as well-behaved as merge sort's.
